

Benchmarking and Studying the LLM-based Agent System in End-to-End Software Development

Zhengran Zeng*
zhengranzeng@stu.pku.edu.cn
Peking University
Beijing, China

Yixin Li*
leason_lyx@stu.pku.edu.cn
Peking University
Beijing, China

Rui Xie
ruixie@pku.edu.cn
Peking University
Beijing, China

Wei Ye
weye@pku.edu.cn
Peking University
Beijing, China

Shikun Zhang
zhangsk@pku.edu.cn
Peking University
Beijing, China

Abstract

The development of LLM-based autonomous agents for end-to-end software development represents a significant paradigm shift in software engineering. However, the scientific evaluation of these systems is hampered by significant challenges, including overly simplistic benchmarks and the difficulty of conducting fair comparisons between different agent architectures due to confounding implementation variables. To address these limitations, we first construct a challenging and dynamically curated E2EDevBench to simulate realistic development scenarios. Second, we propose a hybrid evaluation framework that combines test-case-based functional assessment with fine-grained, LLM-based requirement verification. Using this framework, we conduct a controlled empirical study on three representative agent architectures implemented upon a unified foundation to isolate the impact of workflow design. Our findings reveal that state-of-the-art agents can fulfill approximately 50% of requirements on E2EDevBench, but their success is critically dependent on the architectural strategy for task decomposition and collaboration. Furthermore, our analysis indicates that the primary bottleneck is the omission of requirements and inadequate self-verification. This work provides the community with a more realistic benchmark, a comprehensive evaluation framework, and crucial insights into the current capabilities and core challenges of software development agents, guiding future research toward enhancing requirement comprehension and planning.

1 Introduction

The rapid advancement of Large Language Models (LLMs) in recent years has driven significant progress in the development of LLM-based autonomous agents [37]. These agents are poised to introduce a paradigm shift within the field of software engineering, having already demonstrated capabilities across a range of tasks such as issue fixing, feature implementation, and test generation [1, 25, 33, 35, 36, 41]. Among these applications, the automation of end-to-end software development, which spans the entire lifecycle from initial requirement specification to the delivery of functional software, has emerged as a particularly prominent and critical area of research [25].

With the goal of automating end-to-end software development, the research community has proposed a variety of agent systems,

including ChatDev and MetaGPT [17, 30]. To measure their performance, corresponding benchmarks like SoftwareDev and Commit0 have also emerged [17, 47]. However, despite this growing body of research, the scientific and comprehensive evaluation of these agent systems remains a significant challenge. Specifically, current evaluation approaches often fall short in two significant ways. Firstly, they may present tasks that are overly simplified and lack real-world complexity, a limitation observable in benchmarks such as SoftwareDev [17]. Secondly, in the pursuit of automated and quantitative comparisons, frameworks like Commit0 [47] provide agents with exhaustive test suites or detailed code specifications. Such a setup substantially differs from authentic development workflows, thereby limiting our ability to gauge an agent’s true capabilities in a realistic setting.

Moreover, systematic comparisons between different agent architectures, such as single-agent and multi-agent systems or different workflows, remain limited. A major challenge for fair comparison is the significant variation in how these agents are implemented. Differences in their underlying toolsets and engineering optimizations create confounding variables, making it difficult to perform a fair comparison and analysis. Consequently, a performance gain might be attributed not to a superior architectural design, but rather to better engineering or more powerful tools. Additionally, most existing studies evaluate these agents on relatively simple tasks [17, 27, 30]. This raises a critical question: can their findings be generalized to the more complex and realistic scenarios found in real-world software development?

To address the aforementioned challenges, this study aims to establish a more robust foundation for the evaluation and analysis of agents for end-to-end software development. First, we construct and introduce a more challenging software development dataset, dynamically curated from the latest PyPI projects [4]. This approach mitigates the risk of data leakage and simulates realistic, demanding development scenarios. Second, to meet the complex evaluation demands of end-to-end tasks, we propose a hybrid evaluation methodology. **This methodology integrates test-case-based functional assessment with a fine-grained, LLM-based evaluation of requirement conformance, enabling a more precise measurement of an agent’s performance in fulfilling user needs.** Building on this, to facilitate fair comparisons and investigate the differences between architectures, we implement three representative agent systems based on a unified engineering standard and toolset. This controlled

*Both authors contributed equally to this research.

experimental environment allows us to isolate and study the impact of agent architecture and workflow design on performance.

Through our empirical study, we aim to address several core research questions. Specifically, we seek to: (1) assess the capability of current software development agents in completing realistic end-to-end software development tasks; (2) analyze the impact of different agent workflows (e.g., single-agent versus multi-agent) on performance; and (3) determine the primary causes of agent failure during the development process.

Our research yields several key insights. First, we find that state-of-the-art agents, when powered by the most advanced LLMs, can successfully fulfill approximately 50% of the requirements in our challenging benchmark. Meanwhile, their performance is highly dependent on their architectural design and orchestration strategy. Specifically, effective task decomposition and multi-agent collaboration significantly reduce problem complexity, while poor architectural choices can severely hinder performance. Furthermore, our analysis reveals that the primary reason for failure is not incorrect code implementation, but rather the omission of requirements and inadequate self-verification. This suggests that the core bottleneck lies in the upstream stages of planning and understanding, highlighting the need for further research into requirement comprehension and validation.

The primary contributions of this research can be summarized as follows:

1. **Benchmark:** A more challenging and dynamically updated benchmark dataset for end-to-end software development.
2. **Evaluation Framework:** A comprehensive evaluation framework designed for assessing end-to-end development tasks by combining automated test execution with fine-grained requirement verification.
3. **Approach:** A standardized, open-source implementation of end-to-end development agent system, providing a fair baseline for future comparative research.
4. **Study:** An in-depth empirical study that reveals the capability boundaries and critical bottlenecks of current software development agents.

2 Background and Related Work

2.1 End-to-End (E2E) Software Development

End-to-end software development aims to automate the entire Software Development Life Cycle (SDLC) [25], from initial requirement specification to final product delivery, through a single, autonomous system. The typical task paradigm involves an agent receiving a software requirement specified in natural language (e.g., “develop a 2048 game”) and autonomously generating a complete, functional, and executable code repository.

The recent advancement of LLMs has significantly accelerated the automation of software engineering tasks. For instance, a variety of LLM-driven tools have proven effective for individual stages of the Software Development Life Cycle (SDLC) [18], such as requirements engineering [31], function-level code generation [8, 11], and software testing [35]. Building on these successes, the end-to-end development paradigm aims to integrate these separate steps into a single, seamless pipeline—from initial requirement analysis

to final deployment. Given its great potential and inherent challenges, end-to-end software development is now widely seen as a critical frontier in automation research.

2.2 E2E Software Development Benchmarks

To facilitate the evaluation of automated systems on end-to-end tasks, the research community has proposed several benchmarks. Unlike earlier benchmarks focused on function- or file-level code generation (e.g., HumanEval [11], MBPP [8], APPS [16], and LiveCodeBench [19]), benchmarks for end-to-end development place a greater emphasis on simulating complex, realistic development scenarios.

For instance, the SoftwareDev [17] benchmark features 70 diverse tasks, from arcade games to business systems, that require agents to manage multi-file dependencies and project-level architecture. Similarly, DevEval [21] uses real projects from GitHub to create its tasks, and it evaluates agents across five distinct stages: software design, environment setup, implementation, acceptance testing, and unit testing. ProjectEval [26], which also draws data from SoftwareDev, measures the project-level code generation capabilities of LLMs by providing inputs at different abstraction levels (i.e., from requirements document to code skeleton) and using a predefined test suite for evaluation. In a different approach, Commit0 [47] presents agents with a code skeleton and a requirements document from real GitHub projects, tasking them with completing the code to pass the original tests.

However, existing benchmarks exhibit notable limitations:

1. **Insufficient Challenge:** Many benchmarks, such as SoftwareDev [17], contain tasks that remain at the level of “toy projects.” Meanwhile, the reference implementations for DevEval [21] and ProjectEval [26] are often modest in size (typically 300-400 lines of code), indicating a complexity gap compared to industrial-grade projects.
2. **Limited Evaluation Metrics:** Many studies (e.g., ChatDev [30]) rely on metrics that are often superficial. For instance, “completeness” is typically assessed by checking for code placeholders like “TODO” [30], while “consistency” is often measured by vector similarity between requirements and code [30, 34, 44]. Similarly, “executability” is often just a coarse-grained check to ensure the code runs without crashing [17, 22, 27, 30]. These indirect metrics fail to provide an accurate reflection of software quality.
3. **Compromised Realism in Evaluation:** To resolve interface mismatches between agent-generated code and predefined test cases, benchmarks like DevEval [21], ProjectEval [26], and Commit0 [47] provide test cases or even code skeleton designs as input to the agent. While this approach simplifies evaluation, it deviates significantly from real-world scenarios where only a requirements document is provided.

In light of these shortcomings, our research is dedicated to constructing a benchmark that is both more challenging and more closely aligned with real-world evaluation methodologies. We extract tasks from high-quality PyPI projects and design an evaluation framework that combines automated test migration with objective requirement verification, aiming to measure the true capabilities

of agents more equitably. The specific details of our approach are presented in Section 3.1.

2.3 E2E Software Development Agents

Software Development Agents are autonomous systems based on LLMs that are capable of performing software engineering tasks in a self-directed manner. A typical agent system comprises four core components: Planning, Memory, Perception, and Action [20, 25, 37, 40]. The planning component is responsible for decomposing high-level objectives into executable sub-tasks. The memory component is utilized for storing and retrieving historical information. The perception component is responsible for understanding user instructions and the state of the environment. The action component interacts with the environment by invoking external tools.

Currently, agent architectures applied to end-to-end software development can be broadly categorized into two main classes:

- (1) **Single-Agent:** Exemplified by systems like SWE-Agent [41], the core idea of this approach is to enhance the perception and action capabilities of an individual agent. For instance, SWE-Agent significantly improves an agent’s ability to comprehend and edit complex code by designing a specialized toolset for interacting with a codebase. Similar works, such as ReAct [42], CodeAgent [45] and OpenHands [38], also concentrate on optimizing tool definitions and environmental perception to improve the task execution capabilities of a single agent.
- (2) **Multi-Agent:** Represented by systems like ChatDev [30], AgentVerse [12] and MetaGPT [17], the research focus of this approach lies in task decomposition and workflow orchestration. By simulating collaborative software development models, such as waterfall[32] and agile development [7], these systems break down a complex task into multiple sub-tasks. These sub-tasks are then assigned to agents with distinct roles (e.g., “Designer”, “Programmer”, and “Tester”) for collaborative completion.

Although researchers have proposed various agent architectures, a systematic comparative analysis of these different approaches remains insufficient. A primary challenge is that the performance of agent systems is heavily influenced by their engineering implementation. Research on single-agent systems tends to focus on the perception and action components [38, 41]; consequently, their engineering implementations and toolsets are often more sophisticated. In contrast, the focus of multi-agent systems is on workflow orchestration [17, 30], and their underlying tools may be comparatively simpler. This disparity makes it difficult to conduct direct comparisons that yield reliable conclusions about the inherent merits or demerits of the architectures themselves.

Furthermore, as previously noted, most existing research has been conducted on relatively simple datasets. For example, some studies on multi-agent systems that explore different workflows have compared various orchestration methods under unified experimental conditions [17, 30]. However, the generalizability of their conclusions is limited by the simplicity of the datasets used (Humaneval [11] and MBPP [8]). Therefore, there is a pressing need for research that conducts a fair and in-depth comparison of different agent architectures in more challenging, realistic scenarios, based on unified engineering standards. To address this gap, we implement and evaluate multiple representative agent systems on

a carefully designed benchmark within a controlled experimental environment, enabling us to investigate the intrinsic impact of architectural and workflow design on performance.

3 Study Design

3.1 Benchmark

In order to accurately assess the performance of agents in realistic end-to-end software development scenarios, a more suitable benchmark is required to address the limitations of existing ones. As described in Section 2.2, current benchmarks often fall into two categories. Firstly, datasets like SoftwareDev [17] and SRDD [30] feature tasks of limited complexity, such as implementing small-scale applications or games, which do not adequately represent real-world engineering challenges. Secondly, while more challenging benchmarks like Commit0 [47], DevEval [21], HumanEvo [48] exist, they often require supplementary artifacts such as pre-defined project designs or test cases. This requirement deviates from authentic development workflows, where agents typically operate from high-level requirements alone.

To bridge these gaps, we introduce the **E2EDevBench** (End-to-End Software Development Benchmark), constructed by curating 50 recent, high-quality projects from PyPI [4] with an evaluation framework aimed at addressing the core difficulties in current assessment methodologies. The workflow for our dataset construction and the corresponding evaluation process is illustrated in Figure 1

3.1.1 Dataset Construction. Our dataset construction process is designed to select high-quality, real-world projects with reproducible environments. The process is divided into the following stages:

1. **Source Data Collection and Initial Filtering:** We begin with a large-scale collection of projects from the PyPI [29]. To ensure project quality, we apply a series of filtering rules, retaining only those projects that meet the following criteria: containing more than five Python source files, exhibiting a reasonable code-to-comment ratio, including a test suite, and utilizing pytest [5] as the testing framework.
2. **LLM-based Filtering:** To ensure that projects are both meaningful and reliably executable within an isolated sandbox, we employ an LLM for a second filtering pass. This step systematically excludes projects with trivial functionality, those reliant on hard-to-get dependencies (e.g., requiring specific software or OS utilities), or those that make external API calls. This stage is critical for ensuring the reproducibility of the subsequent evaluation environment.
3. **Execution-based Filtering:** For the projects that pass the filtering stages, we use automated scripts to attempt the construction of a runnable sandbox environment from their provided configuration files (e.g., `requirements.txt`). Subsequently, the script automatically executes the project’s original test suite to confirm both the project’s functional correctness and the completeness of the execution environment.
4. **Sampling and Requirement Generation:** We employ a time-sliced sampling strategy. From the first quarter of 2024 to the first quarter of 2025, we randomly sample 10 standard-compliant projects each quarter, constituting an initial dataset of 50 projects.

This strategy helps mitigate the risk of data leakage and allows the dataset to be continuously updated to reflect the latest development practices. For each finalized project, we generate a requirements document by using an LLM to analyze the source code and README file and produce an initial draft based on a predefined template. This draft is then manually reviewed and revised to ensure accuracy and comprehensiveness.

Ultimately, our construction process yielded a collection of 50 high-quality project development samples, with 10 projects sampled from each quarter between Q1 2024 and Q1 2025. Each instance in the dataset includes: 1) a manually verified requirements document that details the features to be implemented; 2) an interactive sandbox environment for the agent to operate in; 3) and the original source code and test cases, which serve as a reference for our evaluation framework.

As detailed in Table 1, E2EDevBench demonstrates a higher degree of complexity across multiple dimensions compared to existing repository-level benchmarks. In comparison to its counterparts, projects in E2EDevBench exhibit a significant increase in the average number of source files, lines of code (LoC), and test cases, and are accompanied by more detailed requirements documents. These characteristics indicate that E2EDevBench provides a more challenging and realistic testbed for evaluating the end-to-end development capabilities of autonomous agents.

3.1.2 Evaluation Framework. As described in Section 2.2, the evaluation of an agent tasked with developing a project from scratch based solely on a requirements document presents significant challenges. On one hand, the architectural implementation generated by the agent may differ substantially from that of the original project, rendering the original test suite directly unusable. On the other hand, a complete reliance on an “LLM as Judge” [23] for subjective scoring can yield unreliable results due to the inherent biases of the model. To address these challenges, we propose a hybrid evaluation methodology that combines automated test case migration with objective requirement verification.

- 1. Automated Test Case Migration:** After an agent completes the development process and submits its final project, we introduce an independent “Test Migration Agent.” The sole responsibility of this agent is to adapt the original project’s test cases for use with the newly generated project. During this process, we employ technical constraints to strictly prevent the migration agent from modifying the code under evaluation. This approach enables us to leverage high-quality, original test cases for functional verification without providing them as input to the development agent, thereby granting the agent complete freedom in its design and implementation choices.
- 2. Objective Requirement Verification:** After obtaining the execution results from the migrated test cases, we proceed to the requirement verification stage. We provide an LLM with the test results, the agent-generated project code, and the original requirements document. Instead of soliciting a subjective score, we instruct the LLM to perform an objective marking task for each item on the requirements list, classifying it as either “implemented” or “not implemented.” Prior studies have showed that binary classification task to be more reliable than subjective

scoring [9, 49]. To further enhance the stability of our evaluation, we conduct three independent requests for the same assessment; a requirement is marked as definitively “implemented or not implemented” only if all three assessments concur. This process allows us to calculate a precise requirement completion rate as the primary evaluation metric, with the test pass rate serving as a supplementary indicator of functional correctness.

From this hybrid process, we derive our key performance metrics. The functional quality is measured by the *Pass-Rate*, calculated as the ratio of passed test cases (*Passed Count*) to the total. The primary measure of success, however, is the *Implementation Rate (Impl-Rate)*. This metric is computed from the requirement verification stage, representing the proportion of requirements unanimously marked as implemented (*Impl-Req.*) relative to the total number of requirements. The total includes those unanimously marked as not implemented (*Unimpl-Req.*) and those with inconclusive assessments across the three evaluation runs.

3.2 Study Agent

To systematically investigate the impact of different agent workflow designs on the performance of end-to-end software development tasks, we constructed a series of experimental agent systems. This section details our design philosophy and specific implementations.

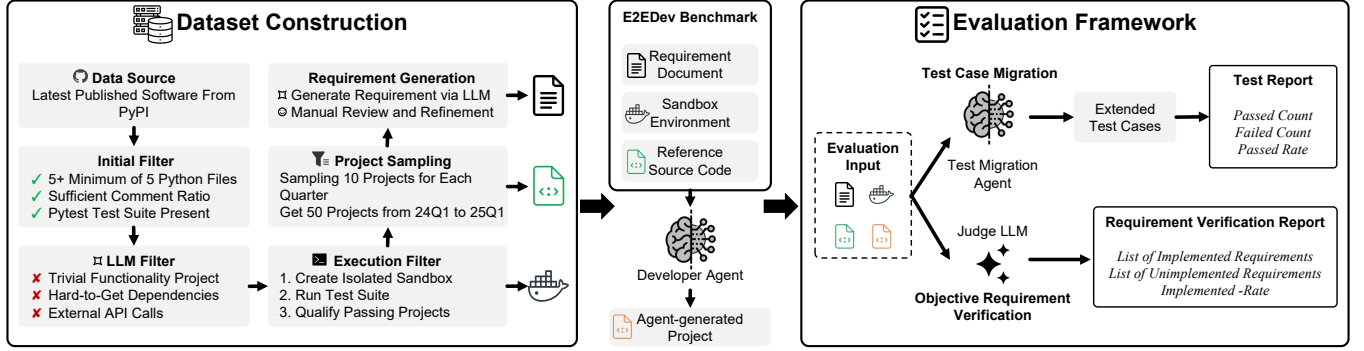
Currently, software development agent systems can be broadly categorized into single-agent and multi-agent architectures. However, conducting a fair comparison between these two paradigms presents significant engineering challenges. As described in Section 2.3, discrepancies in the underlying toolsets, environmental interaction capabilities, and implementation details across different research efforts often act as confounding variables, obscuring the intrinsic merits or demerits of the architectural designs themselves. For instance, in our preliminary explorations with existing multi-agent frameworks (e.g., ChatDev [30], MetaGPT [17]), we observed that their rigid toolchains and limited environmental interaction capabilities, such as an inability to flexibly invoke Bash commands for environment configuration, severely constrained their performance on complex tasks, leading to suboptimal quality in the generated code.

We posit that the core contribution of these frameworks lies not in their specific engineering implementations, but in their conceptualization of multi-agent task workflows, often based on established software development models like the waterfall model[32]. To isolate the influence of implementation-specific factors and facilitate a fair comparison under controlled conditions, we adopted a strategic approach: we selected the representative open-source project SWE-Agent [41] to serve as the unified foundational architecture for all our experimental agents. SWE-Agent provides a mature and flexible toolset, making it an ideal cornerstone for constructing complex agent systems [6]. Building upon this foundation, we focused on implementing and comparing different task workflows, thereby ensuring that observed performance variations could be more directly attributed to the workflow designs themselves. To maintain this focus, our primary goal was to investigate the effectiveness of structured task decomposition. We therefore simplified inter-agent collaboration, modeling it as a sequential handoff of tasks. Inspired by established software development paradigms like

Table 1: The statistical comparison between existing repo-level generation benchmarks.

Benchmark	# Tasks	# Avg. Files	# Avg. LOC	#Avg. Tests	Input Information	Data Leakage
SoftwareDev [17]	70	N/A	N/A	N/A	Requirement	Potential
ProjectDev [27]	14	N/A	N/A	N/A	Requirement	Potential
DevEval [21]	22	4.4	377.8	10.18	Req. + Class Skeleton	Potential
ProjectEval [26]	20	13.2	402.2	14.2	Req. + Class Skeleton /Function Skeleton	Potential
Commit0 [47]	54	205.19	24768.4	730.5	Req. + Class Skeleton	Potential
E2EDevBench	50	19.2	2011.5	119.7	Requirement	Controlled

N/A: Not applicable, as these benchmarks do not have corresponding reference repositories. **Req.**: An abbreviation for Requirement. **Controlled**: Data leakage is can be eliminated by the experiment setting. **LOC**: Lines of Code.


Figure 1: The dataset construction and evaluation workflow of E2EDevBench.

the waterfall model, we designed and implemented three representative agent workflow configurations to explore different divisions of responsibility:

- SDAgent-Single (Software Development Agent).** In this configuration, all development tasks are handled by a single, all-purpose agent. This agent is instructed to follow a linear, self-iterative development process: it first reads the requirements document from the file system, then implements the required functionality in code, creates corresponding tests, and runs these tests. Finally, it reviews the test results and debugs the code, repeating this cycle until all tests pass.
- SDAgent-DT (Developer-Tester).** This configuration simulates a collaborative model with a separation of development and testing duties, utilizing two agents that operate sequentially. The first, a **Developer Agent**, focuses exclusively on development. It is responsible for reading the requirements document and completing the code implementation for all features, but it does not write or execute tests. The second, a **Tester Agent**, it receives the code completed by the Developer Agent, first comprehends the implementation, and then writes test cases based on the original requirements. Subsequently, it executes the tests, and performs any necessary debugging until all tests pass.
- SDAgent-DDT (Designer-Developer-Tester).** This configuration simulating a more classical “design-implement-verify” waterfall workflow with three agents collaborating sequentially. At first, a **Designer Agent**, analyze the requirements document and generate a detailed project design document. Second, a **Developer Agent**, performs the concrete code implementation based

on the design document produced by the Designer Agent and the original requirements. Third, a **Tester Agent**, conducting comprehensive testing and verification of the final implementation.

Experimental Setup: All our experiments were powered by two leading LLMs Gemini-2.5-Pro and Gemini-2.5-Flash [3]. We selected these models for their state-of-the-art capabilities, which serve as a representative foundation for our study [14]. **As our research prioritizes the analysis of workflow design over a comparative LLM benchmark, and to maintain a feasible computational scope, we intentionally limited our experiments to these two models.** In all trials, the generation temperature was set to 0.2, and a **maximum of 200 agent steps was allocated for each task to ensure consistency and termination.** For the processes of benchmark construction and evaluation, we standardized on using Gemini-2.5-Pro to ensure maximum consistency and capability. Specifically, it was the exclusive model used for the dataset annotation, the “Test Case Migration”, and the “LLM as Judge” protocol. Furthermore, the “Test Case Migration” itself was performed by a dedicated agent configured in the Single Agent setup, with the explicit task of migrating test cases.

3.3 Research Questions

This study investigates the following research questions:

- RQ1: How effective is the requirement-driven “LLM as Judge” approach in E2EDevBench?** In this RQ, we evaluate the effectiveness of this requirement-driven approach by comparing

Table 2: Performance analysis of the Original Tests and Extended Tests.

Tests Type	Avg. Passed	Avg. Failed	Pass-Rate	Avg. Covered Lines	Avg. Covered Branches
Original Tests	13.84	1.02	93.14%	156.46	39.83
Extended Tests	15.07	10.19	59.66%	142.17	32.01
All Tests	28.89	11.23	72.01%	188.1	50.5

its assessments against both test suite outcomes and human expert judgments.

- **RQ2: What are the end-to-end development capabilities of current software development agents on the E2EDevBench?** In this RQ, we systematically assess agents with different architectures and base models, focusing on their task success rates, stability, and consistency across multiple trials.
- **RQ3: What are the key challenges and root causes of agent failures in development tasks?** In this RQ, we identify the root causes of failure by analyzing two aspects: identifying common patterns among the unfulfilled requirements, and their common failure modes during key development stages.

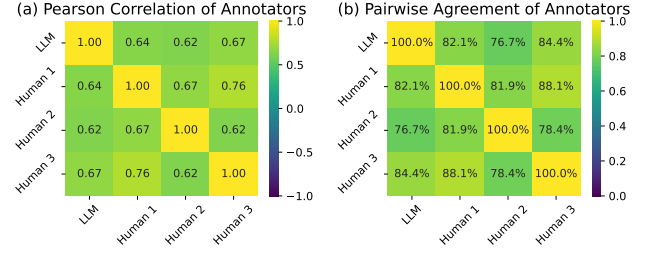
4 Study Results

4.1 RQ1: Effectiveness of Evaluation Framework

To validate the effectiveness of the requirement-driven LLM-as-a-judge evaluation framework in E2EDevBench, we conducted analyses on two key aspects: the comprehensiveness of the test cases and their consistency with human expert assessments.

First, we evaluated the effectiveness of the test case migration scheme, which underpins the comprehensiveness of the evaluation. We performed a statistical analysis of the test run results generated during the RQ2 experiments (detailed in Section 4.2). As shown in Table 2, the test cases are divided into three categories: “Original Tests” (generated by the SDAgent during development), “Extended Tests” (generated by the Test-Migration Agent), and “All Tests” (the union of the first two). The table reports key metrics for execution outcomes (*pass and failure counts, pass-rate*) and code coverage analysis (line coverage count, branch coverage count). The results indicate that, compared to the “Original Tests” *pass-rate* of 93.14%, the “Extended Tests” exhibit a significantly lower *pass-rate* of 59.66%, demonstrating that the “Extended Tests” effectively uncover latent defects missed by the “Original Tests”. Furthermore, the combined test suite (“All Tests”) significantly improves coverage: the average number of covered lines increases from 156.46 to 188.1 (a 20.22% improvement), and covered branches increase from 39.83 to 50.5 (a 26.79% improvement). These results confirm that our test case migration scheme enhances both error-detection capability and coverage breadth, thereby providing a more reliable basis for subsequent requirement-driven evaluations.

Second, we assessed the consistency between the requirement-driven LLM-as-judge and human expert evaluations. We randomly selected 10 agent-generated projects with 546 requirements and invited three computer science graduate students as human experts. Both the LLM (Gemini-2.5-Pro) and the human experts independently annotated whether each requirement had been successfully implemented by the agent. We employed Pearson’s correlation coefficient [28] and pairwise agreement rate [49] to measure annotation

**Figure 2: Analysis of agreement between LLM-as-Judge and human experts.**

agreement, with results presented in Figure ?? . The findings show a high level of agreement between the LLM and all three experts. Notably, the Pearson’s correlation coefficients were all above 0.62, indicating a strong positive correlation between the LLM’s assessments and those of the human experts. Furthermore, the pairwise agreement rate between the LLM and the experts ranged from 76% to 84%, a level comparable to the inter-expert agreement rate of 78% to 88%. This collectively demonstrates that the requirement-driven LLM-as-a-judge evaluation method possesses high reliability and validity.

Conclusion 1:

The test-migration and requirement-driven LLM-as-a-judge evaluation method is effective, it not only significantly increases test coverage and uncovers additional defects by generating new test cases, but also exhibits a high degree of agreement with human expert evaluations, thereby demonstrating the reliability of this evaluation metric.

4.2 RQ2: Performance of SDAgents under E2EDevBench

Building upon the established validity of the E2EDevBench framework, we conducted a comprehensive performance evaluation. Specifically, three agent workflow configurations (SDAgent-Single, SDAgent-DT, and SDAgent-DDT) were each coupled with two model variants (Gemini-2.5-Pro and Gemini-2.5-Flash), with detailed results presented in Table 3. The table shows the average performance of all instances across three key metric categories: requirement completion, test execution performance, and resource-efficiency overhead.

The experimental findings reveal that the SDAgent-DT paired with the Gemini-2.5-Pro model achieved the highest requirement implementation rate of 53.50%. This demonstrates that, when tasked with realistic end-to-end development challenges, the agent can satisfy approximately half of the functional requirements. However, this success is tempered by a significant challenge: the other half of the requirements remained unfulfilled, exposing reliability bottlenecks in the agents’ autonomous development processes. This suggests that, in executing complex tasks independently, current agents lack sufficient robustness and problem-solving capabilities to fully supplant human developers, which is a critical obstacle for future research.

Table 3: Performance of different Agent-LLM combinations on E2EDevBench.

LLM	<i>Impl-Req.</i>	<i>Unimpl-Req.</i>	<i>Impl-Rate.</i>	<i>Passed</i>	<i>Failed</i>	<i>Passed Rate</i>	<i>Tokens Sent (M)</i>	<i>Token Gen (K)</i>	<i>Cost (\$)</i>	<i>Code Lines</i>	<i>Code Files</i>	<i>Steps</i>
SDAgent-Single												
Flash	27.14	23.32	48.46%	42.35	12.96	76.57%	20.90	161.24	6.67	533.10	4.22	152.74
Pro	25.14	21.86	42.97%	28.58	8.92	76.21%	4.96	28.45	6.48	209.02	3.50	80.12
Mean	26.14	22.59	45.72%	35.47	10.94	76.39%	12.93	94.85	6.58	371.06	3.86	116.43
SDAgent-DT												
Flash	26.60	19.30	45.47%	28.04	9.16	75.38%	19.14	192.37	6.22	307.96	4.42	90.60
Pro	31.30	16.14	53.50%	31.34	7.86	79.95%	5.24	49.65	7.05	328.28	6.58	77.36
Mean	28.95	17.72	49.48%	29.69	8.51	77.66%	12.19	121.01	6.64	318.12	5.50	83.98
SDAgent-DDT												
Flash	12.82	31.88	22.63%	25.57	12.49	67.18%	18.27	151.64	5.86	340.86	4.30	108.06
Pro	19.18	27.56	32.79%	26.74	15.52	63.27%	3.60	34.62	4.85	344.92	13.74	86.16
Mean	16.00	29.72	27.71%	26.16	14.00	65.23%	10.94	93.13	5.36	342.89	9.02	97.11

Moreover, agent operation incurs substantial resource overhead. For instance, the top-performing SDAgent-DT + Pro configuration consumed on average 5.24 million *TokensSent* and 49.65 thousand *TokenGen* per project, with an estimated cost of 7.05\$. Such high overhead underscores the urgent need to optimize agent workflows and enhance their resource-utilization efficiency.

Conclusion 2:

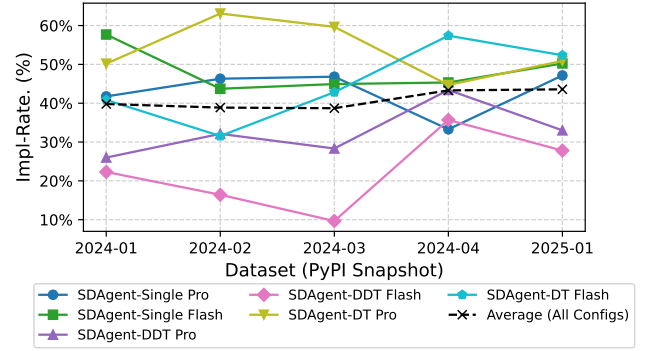
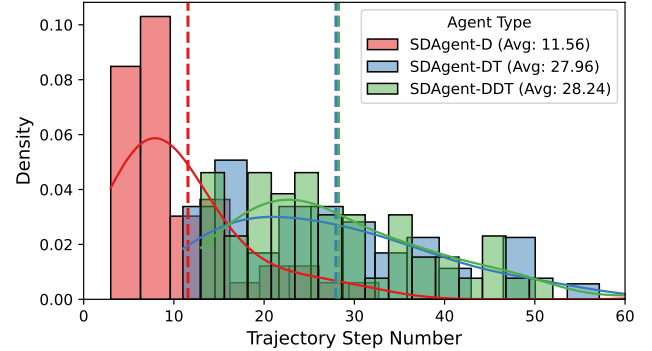
Current advanced software development agents possess the capability to fulfill partial development requirements; nevertheless, their overall performance and cost-efficiency present major areas for improvement.

Although the data in Table 3 demonstrate the agents' remarkable capabilities, it is crucial to verify the authenticity of these results and exclude any potential data leakage. To this end, we assessed the agents' generalization ability using temporally unseen datasets. Specifically, E2EDevBench can be partitioned by project release date, including open-source projects released in 2025-Q1. Given that the Gemini series models employed in our experiments have a knowledge cutoff of January 2025 [13], this dataset can be regarded as completely unseen. Figure3 depicts the *Impl-Rate.* across five time intervals spanning 2024 and 2025-Q1 for different agent-model combinations. The results reveal significant performance stability: when processing the unseen 2025-Q1 data, the agents did not exhibit the anticipated performance drop that might arise from data leakage. Specifically, the average requirement implementation rate in 2025-Q1 (42%) is virtually on par with the 2024 average level (40%). This temporal consistency strongly indicates that the agents' performance reflects genuine generalization capability rather than memorization of the training data.

Conclusion 3:

The agent's performance remained stable on the unseen dataset, thereby excluding the possibility of data leakage and demonstrating its genuine generalization capability in development tasks.

In our comparative analysis of agent architectures, we found that workflow design is a key determinant of performance. As shown in Table 3, the SDAgent-Single achieves the highest requirement implementation rate (49.48%), outperforming SDAgent-Single (45.72%)


Figure 3: Agent performance stability across different time periods.

Figure 4: Distribution of first test writing step by agent.

and far exceeding the lowest-performing SDAgent-DDT (27.71%). To probe the roots of this performance gap, we analyzed each agent's behavioral patterns, especially the coordination between development and testing. Figure 4 depicts the distribution of the step at which each agent first writes a test case. The data indicate that, due to its required develop-then-test workflow, the SDAgent-DT begins writing tests significantly later, on average at step 27.96. In contrast, the SDAgent-Single initiates testing much earlier, at step 11.56 on average, reflecting a highly interleaved code-and-test strategy. We

posit that this seemingly agile iterative mode is the primary cause of SDAgent-*Single*'s suboptimal performance. Frequent context switches between coding and debugging substantially increase the risk of context loss. When errors are detected during testing, the agent tends to fall into a reactive local-fixing loop, deviating from the original global development plan. This "subtask immersion" ultimately leads to neglect of higher-level objectives and failure to satisfy many core requirements. By contrast, the more structured workflow of the SDAgent-DT effectively avoids this trap.

Furthermore, we found that although the SDAgent-DDT adopts a similar develop-then-test strategy to the top-performing SDAgent-DT, its requirement implementation rate remains the lowest. Given that its *Dev* and *Test Agent* are configured identically to those of the SDAgent-*Single*, we hypothesize that the performance bottleneck originates in its *Design Agent*. We attribute this to the fact that the planning blueprint produced by the *Design Agent* negatively impacts subsequent development. Once the *Dev Agent* receives what it perceives as an authoritative implementation plan, it tends to prioritize this plan over direct engagement with the original requirement document, even though the latter is provided as context. If the design plan itself is flawed, such errors are faithfully propagated downstream, resulting in a final product that diverges from the actual requirements. In contrast, the *Dev Agent* within the SDAgent-DT, without a predefined plan, is compelled to interpret and implement directly from the original requirement document. This direct interaction ensures that the development process more faithfully adheres to user needs, thereby achieving a higher requirement implementation rate.

Conclusion 4:

The superior performance of the Dev-Test workflow demonstrates that different workflow designs and agent orchestrations can greatly affect overall performance. A single-agent approach is not necessarily optimal. A well-structured division of tasks and agents can effectively reduce task complexity, whereas an inappropriate decomposition can substantially degrade agent performance.

When comparing the performance of LLMs of different scales, we made a notable observation: on the core metric of requirement fulfillment rate, the relatively weaker Flash model did not exhibit the expected significant performance gap with the more powerful Pro model. For instance, in the SDAgent-*Single*, Flash achieved a fulfillment rate of 48.46%, even outperforming Pro's 42.97%. We hypothesize that this can be attributed to two factors. First, the Flash model's performance is achieved by incurring significantly higher computational costs. Data in Table 3 show that its overhead in *TokensSent* is multiple times that of the Pro model (e.g., 20.90M for Flash vs. 4.96M for Pro in the SDAgent-*Single*). This suggests that Flash model compensates for its weaker single-turn reasoning capabilities through more interaction rounds and trial-and-error. This lengthy and costly exploration is effective sometimes, as each iteration may partially or coincidentally fulfill certain requirements. Consequently, Flash's high fulfillment rate is the cumulative result of successes aggregated through extensive trial-and-error. Second, the Pro model is more efficient in execution (requiring fewer tokens on average), suggesting that the Pro model may be more prone to

prematurely triggering the agent's self-submission condition, judging the task as complete before all requirements are truly met. This pattern of "premature termination" hinders the full exploitation of its advanced capabilities.

Conclusion 5:

Weaker models can match the performance of more capable models to some extent by incurring higher computational overhead. Meanwhile, for more powerful models, the current simplistic agent self-submission mechanism may be constraining their full potential.

In addition, given the inherent stochasticity of LLMs, a common practice is to improve outcomes via "best-of-N" sampling or result aggregation [24, 39]. We therefore evaluated the impact of this randomness on agent performance through three independent runs (N=3, using Gemini-2.5-Pro). To this end, a Venn diagram illustrating the overlap of successful implementations is presented in Figure 5.

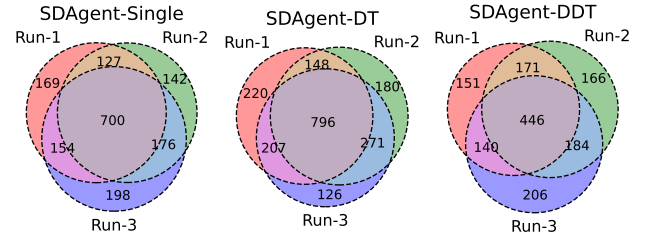


Figure 5: Implemented requirements comparison across 3 runs for each agent.

The Venn diagram reveals a high degree of consistency in the agents' performance. Taking the best-performing SDAgent-DT as an example, a large core set of 796 requirements was jointly fulfilled by all three runs. However, we also observed that a substantial number of requirements (526) were uniquely fulfilled by only one of the three runs. This indicates that a "best-of-N" strategy is a feasible path to improving overall performance by capturing these unique successes. Nevertheless, given the high computational cost of a full agent run, the cost-effectiveness of such an approach becomes a critical concern that requires careful optimization.

Conclusion 6:

While an agent's performance shows high core consistency, multiple runs can capture unique requirement fulfillments, indicating potential for "best-of-N" strategies. However, this approach faces a significant challenge in its high computational cost, making the development of cost-effective aggregation methods a key research direction.

4.3 RQ3: Study of Failure Modes

To investigate the key challenges and root causes of agent task failures, we conducted an in-depth error analysis to understand both the types of unimplemented requirements that occurred and their underlying root causes.

Table 4: A taxonomy of unimplemented requirements and their distribution.

Category (Distribution)	Description
1. Missing Component or Feature (32.4%)	The required functionality is entirely absent from the codebase.
2. Incomplete Implementation (18.1%) 2.1 Placeholder or Stub Implementation (6.4%) 2.2 Incomplete Logic or Behavior (11.7%)	The feature is partially present but fails to meet the full scope of the requirement. A structural element is defined, but its implementation is empty or non-functional. The implementation handles the primary use case but is missing required sub-features or edge case handling.
3. Incorrect Implementation (34.1%) 3.1 Flawed Core Logic (17.7%) 3.2 Incorrect Output or Data Formatting (6.0%) 3.3 Incorrect API or Signature (10.4%)	The feature has been implemented in its entirety, but it behaves incorrectly or produces the wrong results. The fundamental algorithm or business logic used to implement the feature is incorrect. The final output does not conform to the specified format. (e.g., incorrect string formatting, data structure, or file content). The implementation violates the required software architecture or public-facing contracts, such as incorrect class inheritance.
4. Dependent or Upstream Failure (15.3%)	This is a cascading failure where the component fails due to its reliance on a defective or unimplemented upstream dependency.

To start with, drawing upon prior work [10], we established an error taxonomy developed based on Grounded Theory [15] through an iterative manual analysis of unimplemented requirements. The full taxonomy is presented in Table 4. Specifically, we classify unimplemented requirements into four high-level categories: Missing Component or Feature, where functionality is entirely absent; Incomplete Implementation, where a feature is only partially realized; Incorrect Implementation, where the logic is flawed; and Dependent or Upstream Failure, where the error stems from other system parts.

For a large-scale quantitative analysis, we randomly sampled 1,000 unimplemented requirements from 50 projects in the Pro model experiments across three agents' configurations. We employed an "LLM pre-annotation, then human refine" methodology to classify these samples, ensuring the accuracy and reliability of the annotations. Table 4 also illustrates the distribution of each unimplemented requirement type. The analysis reveals a key pattern: failures stem more from "omission and validation failure" than from "implementation errors." The largest category of failures was due to components or features being completely omitted (32.4%), followed by requirements being only partially implemented (21.7%). Both of these error types can be attributed to the LLM's inadequate comprehension of the requirements. Furthermore, 15.3% of failures were the result of cascading failures from upstream tasks. In contrast, genuine implementation errors accounted for 34.1% of the failures. Aggregating these categories reveals that the vast majority of failures are not implementation errors at the code level, but rather omissions during the planning and execution process.

Conclusion 7:

The primary failure mode for agents is "omission and validation failure," not "implementation error." Most requirement failures stem from requirements being completely overlooked, partially implemented, or blocked by upstream errors.

Next, we traced the root causes of failure from the perspective of agent capabilities. Employing the same annotation method, we

attributed failures to different stages of the agent's execution trace. Using the same annotation method, we classified failures into three distinct categories based on their origin, as presented in Table 5. These categories are: **Task Planning**, where errors occur before implementation begins, such as omitting or misinterpreting requirements; **Task Execution**, where failures happen during the development process, e.g., technical roadblocks, context forgetting [43, 46]; and **Task Verification**, where the agent fails to properly assess its own work, (e.g., creating insufficient test cases and premature submission).

The results in Table 5, clearly indicate that "Task Planning" is the primary bottleneck in current agent systems, accounting for 55.8% of all issues. Specifically, the most prevalent problems are *requirements being directly overlooked* (27.9%) or *misunderstood and oversimplified* (22.2%) during the planning phase. In stark contrast, direct failures genuinely caused by insufficient core LLM capabilities (e.g., the inability to solve a specific technical challenge) account for only a minimal fraction (5.6%). However, these few core capability failures have a disproportionately large downstream impact, potentially causing the subsequent 21% of requirements to be blocked or invalidated due to dependencies. This reveals that in a complex task chain, the failure of a few critical nodes can trigger a widespread cascading effect.

Conclusion 8:

The core constraint of current agent systems is "planning and comprehension". The vast majority of failures stem from the omission or misinterpretation of requirements during the planning phase. This suggests that enhancing an agent's ability to comprehend, decompose, and validate requirements is a more critical research direction.

5 Implications and Discussions

Our empirical study yields several important implications for the future of LLM-based software development agents.

Requirement-driven evaluation is effective. Our study demonstrates that the proposed hybrid methodology, combining test-case

Table 5: A taxonomy of failure root causes and their distribution.

Category (Distribution)	Description
1. Task Planning (55.8%) 1.1 Requirement Omission (27.9%) 1.2 Requirement Misinterpretation (22.2%) 1.3 Architectural Design Flaw (5.6%)	The agent entirely overlooks a requirement, making no attempt to implement it. The agent’s understanding of the requirement is flawed (e.g., misinterpreting logic, specs, or constraints), leading to an implementation that fails to meet the specification. A flawed architectural decision made early on renders subsequent tasks impossible to implement correctly.
2. Task Execution (38.6%) 2.1 Technical Capability Bottleneck (3.2%) 2.2 Context & Task Forgetting (1.9%) 2.3 Superficial Implementation (12.5%) 2.4 Dependent Feature Failure (21.0%)	The agent fails because it cannot overcome the task’s inherent technical complexity. The agent loses track of key information or planned tasks over long interactions, leading to omitted actions or a logically broken implementation. The agent implements only the core ‘happy path’ of a requirement, ignoring necessary edge cases, error handling, and robustness requirements. A cascading failure caused by the incorrect implementation of an upstream feature it depends on.
3. Task Verification (5.7%) 3.1 Inadequate Verification (5.7%)	The agent’s own tests are too simplistic or incomplete to detect flaws in its implementation.

migration with a requirement-driven “LLM as Judge,” proves to be highly effective. It not only increases test coverage to uncover hidden defects but also shows a strong correlation with human expert judgment (Conclusion 1). This implies that future research should adopt more holistic evaluation frameworks that measure not just functional correctness but also the granular fulfillment of user requirements, thereby providing a more reliable measure of an agent’s true capabilities.

Autonomous software development is a promising reality. Our findings confirm that end-to-end software development by LLM-based agents is feasible. State-of-the-art agents demonstrate a genuine ability to generalize and complete a substantial portion of complex, real-world development tasks on unseen data (Conclusion 2 & 3). This provides strong evidence that this research direction holds significant promise. However, the path to full autonomy is still fraught with challenges, particularly concerning the reliability and computational cost of these systems. This suggests that while the long-term vision is achievable, near-term research should focus on enhancing robustness and efficiency to make these agents practical for real-world deployment.

Agent orchestration is a critical performance factor. This research highlights that agent architecture is important. A well-designed multi-agent workflow, such as the Developer-Tester configuration, can significantly reduce task complexity and outperform a single-agent approach (Conclusion 4), while poor design can hamstring even the most powerful models. This finding implies that future work should explore and optimize agent collaboration patterns and task decomposition strategies as an important direction for improving performance.

The core bottleneck is comprehension. A key finding from our analysis is that the principal bottleneck for agents is planning and comprehension. The vast majority of failures stem not from an inability to write correct code, but from the omission, misinterpretation, or inadequate verification of requirements (Conclusion 7 & 8). This indicates that the core challenges are cognitive—related to

planning and understanding—rather than technical execution. This has profound implications for the research community: a greater emphasis must be placed on enhancing an agent’s ability to comprehend specifications, construct complete plans, and perform rigorous self-validation. Improving the agent’s “requirement engineering” skills may yield greater returns than simply improving its code generation proficiency.

6 Threats to Validity

Internal Validity. The main threat arises from potential implementation errors in our unified agent framework, which could create confounding variables in our architectural comparison. We mitigated this by performing careful code reviews and making our entire experimental setup publicly available for scrutiny and replication [2].

External Validity. This threat arises from our choice of LLMs and the quality of our benchmark. To mitigate it, we selected two SOTA models, Gemini-2.5-Pro and -Flash. As our research focuses on the impact of agent architecture rather than a comparison of LLM capabilities, we contend these models are representative. Next, to ensure the benchmark’s quality, all task requirements were rigorously revised by human experts, guaranteeing their quality.

Construct Validity. A significant threat is the reliability of our “LLM as Judge” component for evaluating requirement fulfillment. To address this, we validated our automated evaluation metric by demonstrating its high consistency with assessments performed by human experts, thereby ensuring that our framework accurately measures an agent’s ability to meet user needs.

7 Conclusion

In this paper, we conducted a systematic evaluation of LLM-based agents on a new, challenging end-to-end software development benchmark. Our study, facilitated by a hybrid evaluation framework and controlled agent implementations, reveals that while current agents can fulfill approximately half of the requirements (50%), their

success is critically determined by their architectural design. The primary bottleneck is not in the execution phase but rather failures during the planning and verification phases, such as the omission or misinterpretation of requirements. These findings strongly suggest that future research must prioritize enhancing agents' capabilities in requirement comprehension and self-assessment. Our work provides a benchmark and framework to guide and measure progress in this vital direction.

References

- [1] [n. d.]. Aider Documentation — aider.chat. <https://aider.chat/docs/>. [Accessed 18-07-2025].
- [2] [n. d.]. *E2EDevBench*. <https://anonymous.4open.science/r/E2EDevStudy-6C70>
- [3] [n. d.]. Gemini 2.5: Our most intelligent AI model — blog.google. <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/gemini-2-5-pro>. [Accessed 18-07-2025].
- [4] [n. d.]. PyPI The Python Package Index — pypi.org. <https://pypi.org/>. [Accessed 18-07-2025].
- [5] [n. d.]. pytest documentation — docs.pytest.org. <https://docs.pytest.org/en/stable/index.html>. [Accessed 18-07-2025].
- [6] [n. d.]. SWE-bench Leaderboards — swebench.com. <https://www.swebench.com/>. [Accessed 16-07-2025].
- [7] Mikio Aoyama. 1997. Agile Software Process model. In *21st International Computer Software and Applications Conference (COMPSAC '97)*, 11-15 August 1997, Washington, DC, USA. IEEE Computer Society, 454–459. doi:10.1109/COMPSAC.1997.625042
- [8] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 <https://arxiv.org/abs/2108.07732>
- [9] Anya Belz, Craig Thomson, Ehud Reiter, and Simon Mille. 2023. Non-Repeatable Experiments and Non-Reproducible Results: The Reproducibility Crisis in Human Evaluation in NLP. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 3676–3687. doi:10.18653/v1/2023.findings-acl.226
- [10] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya G. Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? *CoRR* abs/2503.13657 (2025). arXiv:2503.13657 doi:10.48550/ARXIV.2503.13657
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [12] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=EHg5GDnyq1>
- [13] gemini. [n. d.]. Gemini 2.5 Pro Model Card. <https://storage.googleapis.com/model-cards/documents/gemini-2.5-pro.pdf>. [Accessed 18-07-2025].
- [14] Gemini. [n. d.]. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf. [Accessed 18-07-2025].
- [15] Barney Glaser and Anselm Strauss. 2017. *Discovery of grounded theory: Strategies for qualitative research*. Routledge.
- [16] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c24cd76e1ce41366a4bbe8a49b02a028-Abstract-round2.html>
- [17] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VtmBAGCN7o>
- [18] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024), 220:1–220:79. doi:10.1145/3695988
- [19] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=chfJYJC3iL>
- [20] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. *CoRR* abs/2408.02479 (2024). arXiv:2408.02479 doi:10.48550/ARXIV.2408.02479
- [21] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2025. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, 7511–7531. <https://aclanthology.org/2025.coling-main.502/>
- [22] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2025. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, 7511–7531. <https://aclanthology.org/2025.coling-main.502/>
- [23] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. 2024. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. *CoRR* abs/2411.16594 (2024). arXiv:2411.16594 doi:10.48550/ARXIV.2411.16594
- [24] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. *CoRR* abs/2203.07814 (2022). arXiv:2203.07814 doi:10.48550/ARXIV.2203.07814
- [25] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large Language Model-Based Agents for Software Engineering: A Survey. *CoRR* abs/2409.02977 (2024). arXiv:2409.02977 doi:10.48550/ARXIV.2409.02977
- [26] Kaiyuan Liu, Youcheng Pan, Jing Li, Daojing He, Yang Xiang, Yexing Du, and Tianrun Gao. 2025. ProjectEval: A Benchmark for Programming Agents Automated Evaluation on Project-Level Code Generation. *CoRR* abs/2503.07010 (2025). arXiv:2503.07010 doi:10.48550/ARXIV.2503.07010
- [27] Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. 2025. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. In *IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering, Forge@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*. IEEE, 156–167. doi:10.1109/Forge66646.2025.00026
- [28] Karl Pearson. 1895. Note on Regression and Inheritance in the Case of Two Parents. *Proceedings of the Royal Society of London* 58 (Jan. 1895), 240–242. doi:10.1098/rspl.1895.0041
- [29] PyPI. [n. d.]. BigQuery Datasets - PyPI Docs — docs.pypi.org. <https://docs.pypi.org/api/bigquery/>. [Accessed 15-07-2025].
- [30] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development.

- In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 15174–15186. doi:10.18653/V1/2024.ACL-LONG.810
- [31] Krishna Ronanki, Beatriz Cabrero Daniel, Jennifer Horkoff, and Christian Berger. 2023. Requirements Engineering using Generative AI: Prompts and Prompting Patterns. *CoRR* abs/2311.03832 (2023). arXiv:2311.03832 doi:10.48550/ARXIV.2311.03832
- [32] W. W. Royce. 1987. Managing the Development of Large Software Systems: Concepts and Techniques. In *Proceedings, 9th International Conference on Software Engineering, Monterey, California, USA, March 30 - April 2, 1987*, William E. Riddle, Robert M. Balzer, and Kouichi Kishida (Eds.). ACM Press, 328–339. <http://dl.acm.org/citation.cfm?id=41801>
- [33] André Silva and Martin Monperrus. 2025. RepairBench: Leaderboard of Frontier Models for Program Repair. In *IEEE/ACM International Workshop on Large Language Models for Code, LLM4Code@ICSE 2025, Ottawa, ON, Canada, May 3, 2025*. IEEE, 9–16. doi:10.1109/LLM4CODE66737.2025.00006
- [34] Aidar Valeev, Roman Garaev, Vadim Lomshakov, Irina Piontkovskaya, Vladimir Ivanov, and Israel Adewuyi. 2025. YABLoCo: Yet Another Benchmark for Long Context Code Generation. In *IEEE/ACM International Workshop on Large Language Models for Code, LLM4Code@ICSE 2025, Ottawa, ON, Canada, May 3, 2025*. IEEE, 88–95. doi:10.1109/LLM4CODE66737.2025.00016
- [35] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Trans. Software Eng.* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208
- [36] Kaixin Wang, Tianlin Li, Xiaoyu Zhang, Chong Wang, Weisong Sun, Yang Liu, and Bin Shi. 2025. Software Development Life Cycle Perspective: A Survey of Benchmarks for Code Large Language Models and Agents. *CoRR* abs/2505.05283 (2025). arXiv:2505.05283 doi:10.48550/ARXIV.2505.05283
- [37] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers Comput. Sci.* 18, 6 (2024), 186345. doi:10.1007/S11704-024-40231-1
- [38] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=OJd3ayDDoF>
- [39] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=1PL1NIMMrw>
- [40] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. 2025. The rise and potential of large language model based agents: a survey. *Sci. China Inf. Sci.* 68, 2 (2025). doi:10.1007/S11432-024-4222-0
- [41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X
- [43] Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, Guohao Dai, Shengen Yan, and Yu Wang. 2024. LV-Eval: A Balanced Long-Context Benchmark with 5 Length Levels Up to 256K. *CoRR* abs/2402.05136 (2024). arXiv:2402.05136 doi:10.48550/ARXIV.2402.05136
- [44] Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Wei Li, Yafen Yao, Yongshun Gong, Xiaolin Chen, Bei Guan, Zhiguang Yang, Yongji Wang, Qianxiang Wang, and Lizhen Cui. 2024. CodeS: Natural Language to Code Repository via Multi-Layer Sketch. *CoRR* abs/2403.16443 (2024). arXiv:2403.16443 doi:10.48550/ARXIV.2403.16443
- [45] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 13643–13658. doi:10.18653/V1/2024.ACL-LONG.737
- [46] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. infyBench: Extending Long Context Evaluation Beyond 100K Tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 15262–15277. doi:10.18653/V1/2024.ACL-LONG.814
- [47] Wenting Zhao, Nan Jiang, Celine Lee, Justin T. Chiu, Claire Cardie, Matthias Gallé, and Alexander M. Rush. 2025. Commit0: Library Generation from Scratch. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=MMwaQEVsAg>
- [48] Dewu Zheng, Yanlin Wang, Ensheng Shi, Ruikai Zhang, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. 2025. HumanEvo: An Evolution-Aware Benchmark for More Realistic Evaluation of Repository-Level Code Generation. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1372–1384. doi:10.1109/ICSE55347.2025.00228
- [49] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html